

Program Representation, Language Translation, and Execution

JAHEZIA

**College of Science
Northern Border University**



Programs as Data

Programs can use the output of other programs as their input, and they can also interact with one another in many ways. This idea is central to software engineering and operating systems, allowing modular design and supporting more complex functionality.

Several mechanisms allow programs to interact with one another:

1. Pipes and Redirection: In command-line environments, one program's output can be **piped** (`|`) directly into another's input. For example, `ls -l | grep .txt` lists files and then filters the results for ".txt". Output can also be redirected (`>`) to a file for another program to use later.

2. Subprocesses: A program can start another program as a subprocess and communicate with it through its standard input, output, and error streams. Many languages provide tools for this, such as Python's subprocess module.

3. Inter-Process Communication (IPC): More advanced or long-lasting communication can be handled through mechanisms like:

- **Named Pipes:** Similar to regular pipes but accessible via the file system, letting unrelated processes exchange data.
- **Sockets:** Allow processes to communicate locally or over a network, even between different machines.
- **Shared Memory:** Lets processes access a common memory region for very fast data sharing.
- **Message Queues:** Enable asynchronous message passing between processes.
- **APIs and Libraries:** Programs often communicate by calling functions exposed through APIs, whether from system libraries, third-party modules, or remote web services.

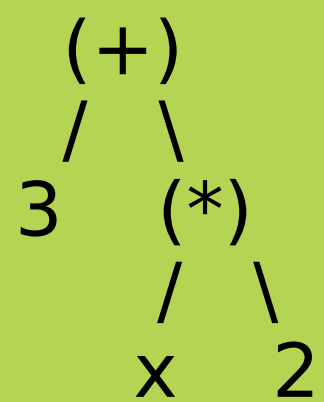
4. APIs and Libraries: Programs often communicate by calling functions exposed through APIs, whether from system libraries, third-party modules, or remote web services.

Abstract Syntax Tree(ASTs)

An Abstract Syntax Tree (AST) is a tree-structured representation of source code that captures its essential syntax while ignoring details like punctuation. Each node represents a construct—such as variables, operators, or function calls—and the branches show how these elements relate. ASTs are central to compilers, linters, and other tools for analyzing, transforming, and optimizing code.

Characteristics:

- Nodes represent language constructs (e.g., expressions, statements)
- Abstracts away concrete syntax (no parentheses, punctuation)
- Used internally by compilers, interpreters, and analysis tools



Example: Expression `3 + x * 2`

Key Characteristics of ASTs

- **Abstract representation:** ASTs strip away unnecessary syntax (like commas or parentheses) and keep only the essential program structure.
- **Hierarchical structure:** The tree form shows how code elements relate—e.g., a function call node may contain child nodes for the function name and its arguments.
- **Concise and meaningful:** By removing low-level parsing details, ASTs are more compact and easier to analyze than full parse trees.
- **Typed nodes:** Each node corresponds to a specific construct, such as a literal value, a function call, or an assignment expression.

How ASTs Are Used

- **Compilers:** They parse code into an AST, optimize it, and then generate machine code from it.
- **Code analysis:** Static analysis tools inspect ASTs to check syntax, detect patterns, or measure code usage.
- **Code transformation:** Formatters and similar tools convert code to an AST and rewrite it in a consistent style.
- **Development tools:** Build systems and modern tooling (e.g., Webpack, Parcel) use ASTs to read, modify, and bundle code.

Tools That Process Code

Concepts of Interpreter, expression optimizer, and documentation generator.

Interpreter : An interpreter is a program that directly executes instructions written in a programming or scripting language, line by line, without requiring them to be compiled into machine code beforehand.

- Directly executes AST or bytecode
- Steps through operations without generating machine code

Expression Optimizer : An expression optimizer is a component, often within a compiler or interpreter, that improves the efficiency (speed or memory usage) of code or mathematical expressions.

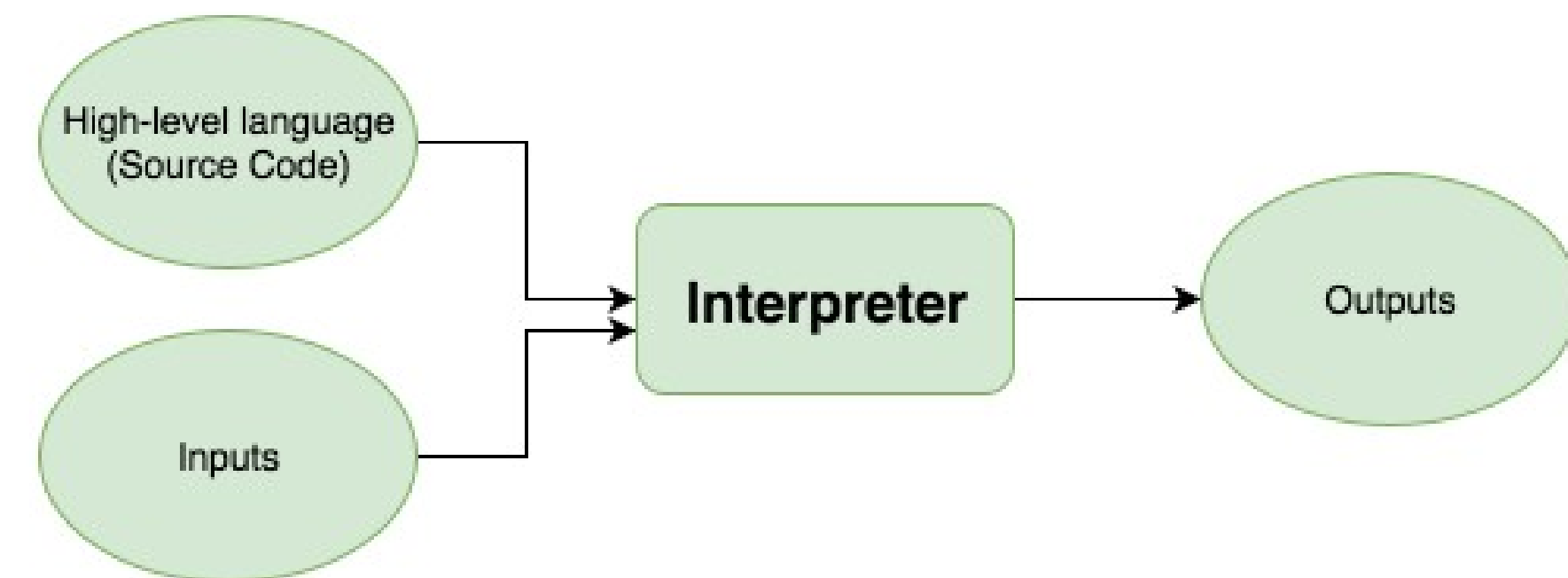
- Simplifies or transforms code
- Example: $3 + 4 \rightarrow 7$, $x * 1 \rightarrow x$

Documentation Generator : A documentation generator is a tool that processes source code and associated comments to produce human-readable documentation.

- Parses comments & syntax $\rightarrow$ produces human-readable docs

Interpreter :

- **Function:** Translates and executes source code one statement at a time during runtime.
- **Process:** It reads a line, analyzes it (lexical and syntax analysis), and executes it immediately. This process stops at the first error encountered.
- **Characteristics:**
 - > **Portability:** Often more portable than compiled code, as the interpreter itself is tailored to the specific machine architecture.
 - > **Debugging:** Simplifies debugging because errors are reported immediately, line by line.
 - > **Performance:** Generally slower than compiled programs because the translation happens on the fly for every execution instance. Modern interpreters often use Just-In-Time (JIT) compilation to mitigate this.



Expression Optimizer :

- **Function:** A phase in language processing (compiler or interpreter design) focused on improving the quality and efficiency of the code, typically in terms of speed, memory usage, or storage size.
- **Process:** Optimizers apply various transformations (e.g., constant folding, dead code elimination, loop optimization) to make the code run more efficiently without changing its overall function.

- **Characteristics:**

It can operate on an intermediate representation of the code before final execution or code generation.

Optimizations can be "local" (within a basic block of code) or "global" (across entire functions or modules).

Expression optimizers specifically target individual expressions, for instance, replacing $3.14 > 2$ and $3.15 < 1$ with its boolean result at compile time if the values are constant.

Documentation Generator :

- **Function:** A software tool that automatically generates structured, often hyperlinked, human-readable documentation from source code and embedded comments (like Javadoc or Doxygen).
- **Process:** It parses the source code to extract structural information (classes, functions, parameters, data types) and specially formatted comments. It then formats this information into various output formats (e.g., HTML, PDF, Markdown).

- **Characteristics:**

Automation: Reduces the manual effort required to keep documentation synchronized with code changes.

Standardization: Enforces a consistent format and style for documentation across a project.

Integration: Becomes a standard part of the software development lifecycle and Continuous Integration/Continuous Deployment (CI/CD) pipelines.

Syntax and parsing vs. semantics and evaluation.

Syntax & Parsing (Form)

1. Syntax defines the rules for how code must be written—its structure and valid symbol arrangements.

2. Parsing checks the code against these rules and builds a structural representation (often an AST).

Focuses on:

- Structure
- Grammar
- Correct arrangement of tokens
- Errors detected: Syntax errors (e.g., missing parentheses, misplaced keywords).

Semantics and Evaluation

1. **Semantics:** defines the **meaning of language constructs**. It determines what a syntactically correct program **actually does**, including type checking, variable scope, and the interpretation of expressions and statements. A program can be syntactically valid but semantically incorrect (e.g., adding a string to an integer in a strongly-typed language).

2. **Evaluation:** (or execution) is the **process of carrying out the program's meaning**. After semantic analysis ensures correctness, evaluation interprets instructions, manipulates data, and manages program flow to produce the output or side effects.

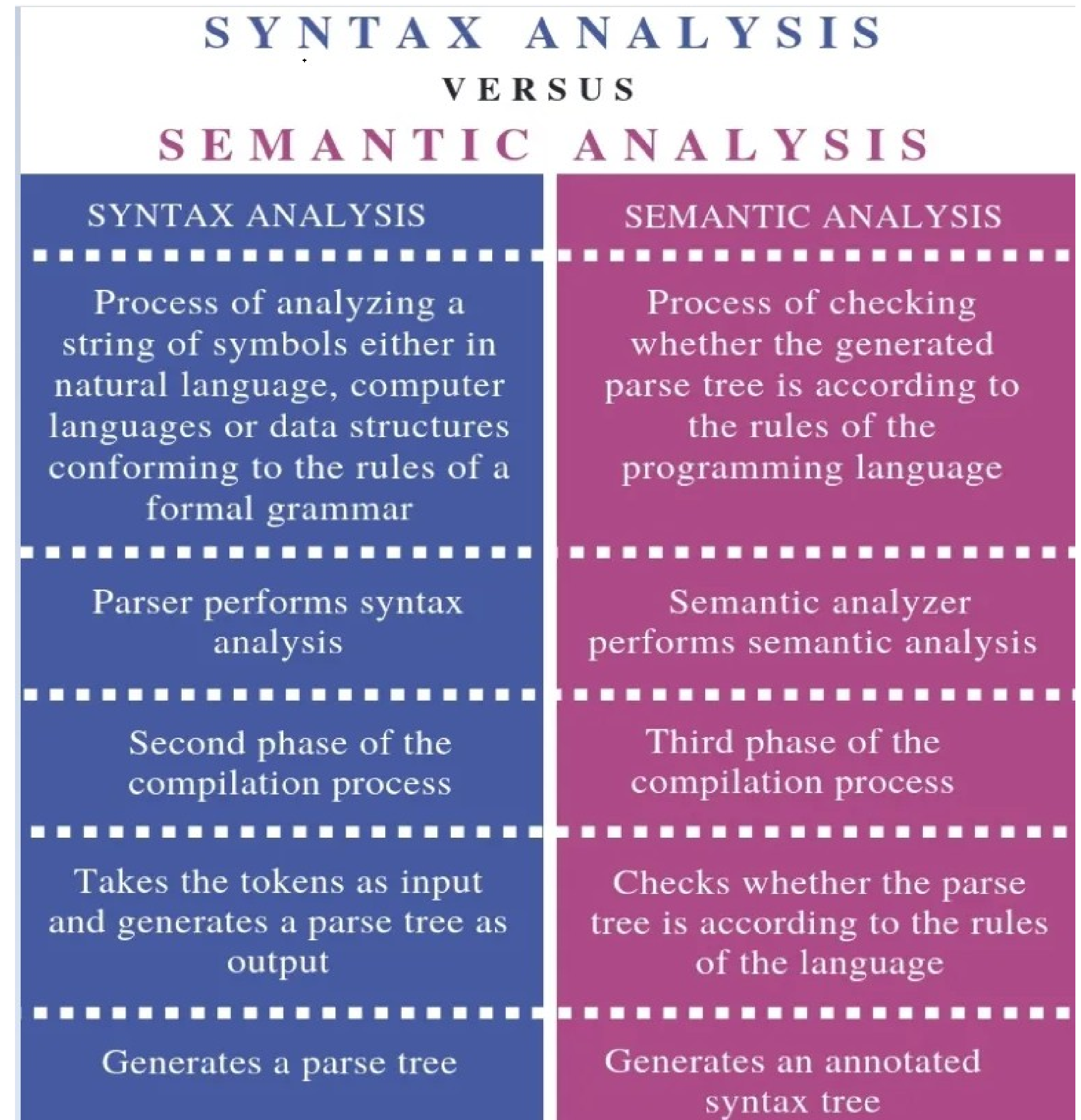
Focuses on :

- **Semantics defines meaning** – what syntactically correct code actually does.
- **Semantics checks correctness** – types, variable scope, and valid operations.
- **Evaluation executes code** – performs computations and manages program flow.
- **Evaluation produces results** – outputs values or side effects according to semantics.

Syntax and parsing vs. semantics and evaluation.

Relationship and Distinction:

- Parsing precedes semantic analysis and evaluation. A program must first be syntactically valid before its meaning can be determined and subsequently executed.
- Syntax focuses on form and structure, while semantics focuses on meaning and behavior.
- Parsing builds a structural representation; evaluation executes the actions described by that structure and its associated meaning.
- Syntax errors are typically caught during parsing, while semantic errors (like type mismatches or logical flaws) are identified during semantic analysis or at runtime during evaluation.



Low-level runtime representation of core language constructs.

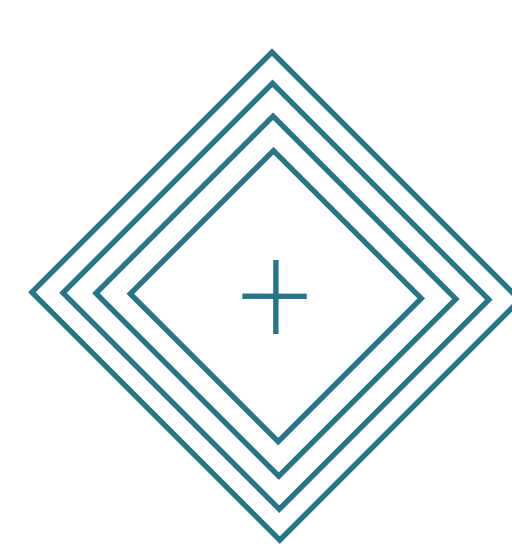
Low-level runtime representation of core language constructs refers to how the fundamental elements of a programming language are represented and managed in memory during program execution. It's the bridge between high-level code and the machine-level operations that actually run.

Examples:

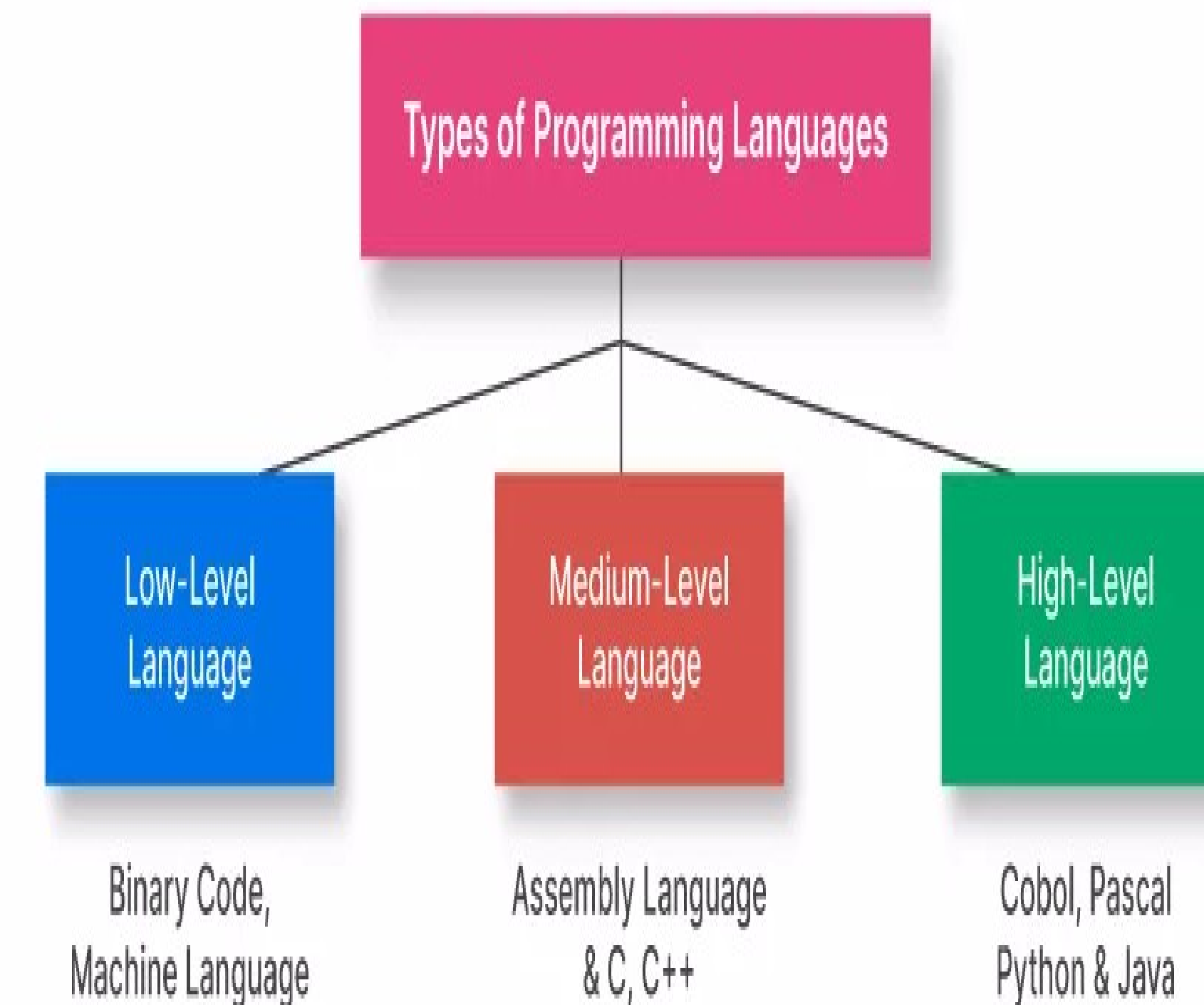
- **Variables** → memory addresses
- **Functions** → stack frames, return addresses
- **Objects/structs** → contiguous or pointer-linked memory blocks
- **Arrays** → fixed-size blocks indexed by offset
- **Control flow** → instruction pointers, jump tables

Key points:

- **Variables and Data** – stored in memory (stack, heap, registers) with specific formats depending on type.
- **Control Structures** – loops, conditionals, and function calls are represented as sequences of instructions or control flow constructs at runtime.
- **Functions/Procedures** – activation records or call frames track local variables, parameters, and return addresses.
- **Expressions and Operations** – evaluated according to runtime rules; often compiled to machine instructions or intermediate code.
-

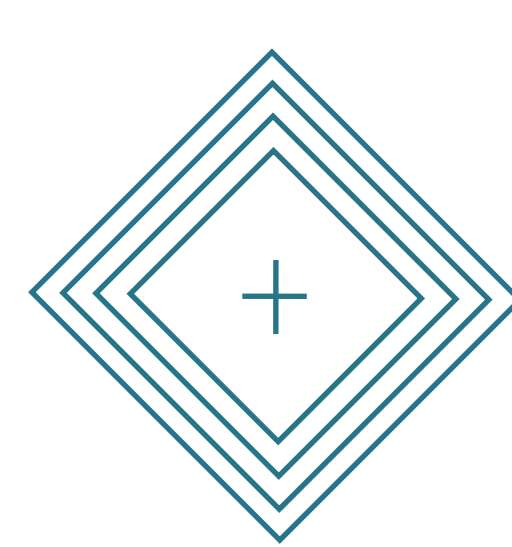


Types of Programming Languages

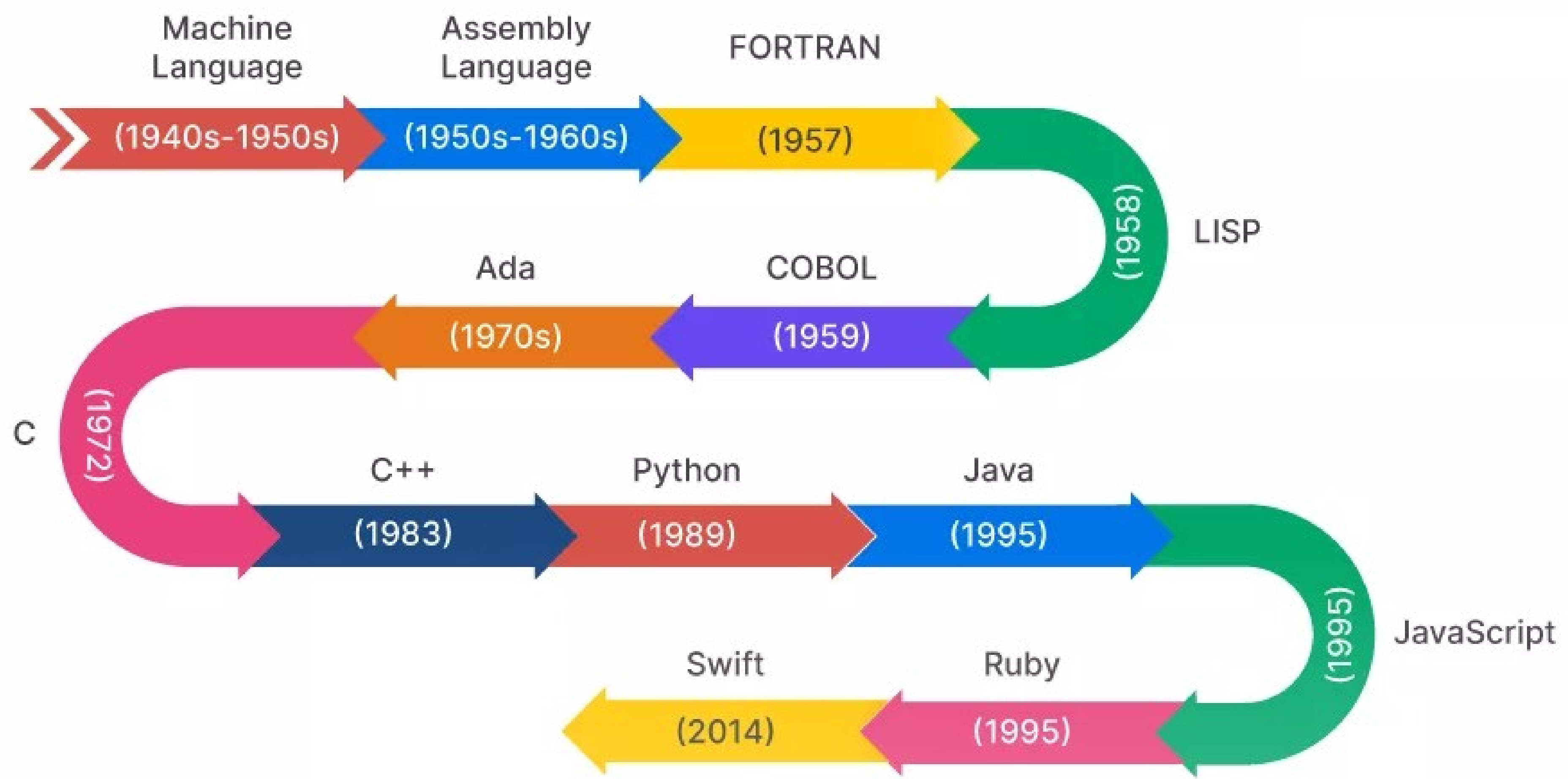


Difference Between High-level And Low-level Programming Languages

Low-level Languages	High-level Languages
• Easy to understand for machine, but less human-readable • Not Portable • Higher execution speed • Uses binary code and mnemonics • Examples: Assembly language, Machine language	• Human-readable and easier to understand and use. • Portable • Comparatively lower execution speed • Uses syntax and semantics • Examples: Java language, Python, C++



The Evolution Of High-level And Low-level Languages

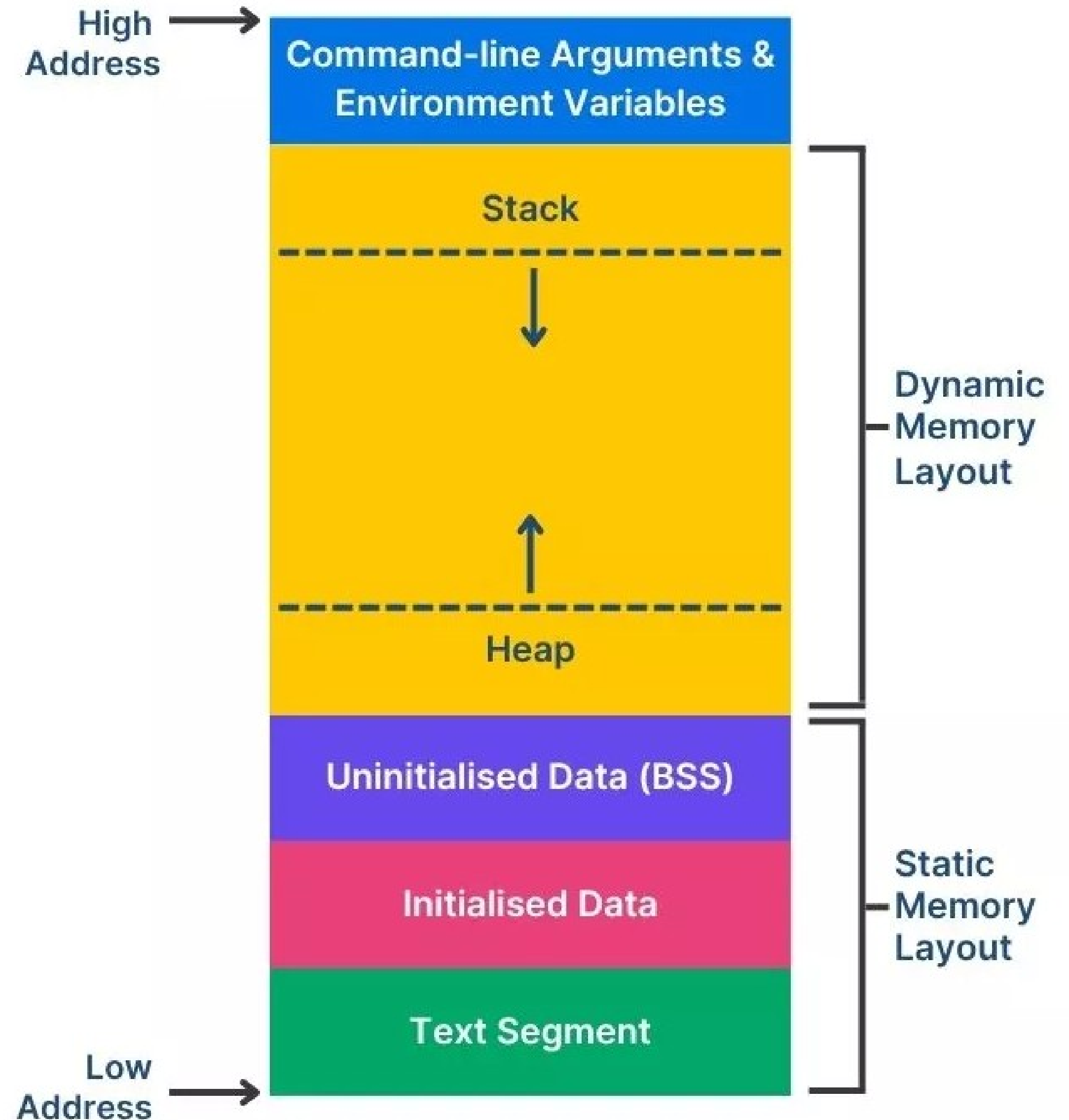


Organization of memory into global data, text, heap, and stack sections by programming language implementations.

Programming language implementations usually divide a program's memory into multiple separate segments or sections, each designed to handle specific kinds of data or executable code while the program runs.

1. Text Segment (or Code) : This segment contains the program's compiled machine instructions and is usually marked read-only to protect the program's logic from being altered during execution.

- > Contains compiled machine instructions of the program.
- > Typically read-only to prevent accidental modification and enable optimizations such as:
Instruction caching
- > Sharing text across processes (common for shared libraries)



Global Data Section (Static Storage Area)

This segment holds global and static variables that are initialized at compile time. It is further subdivided into two major subsections:

1. **.data** — Initialized Global/Static Variables
2. **.bss** — Uninitialized Global/Static Variables

1. **.data** — Initialized Global/Static Variables

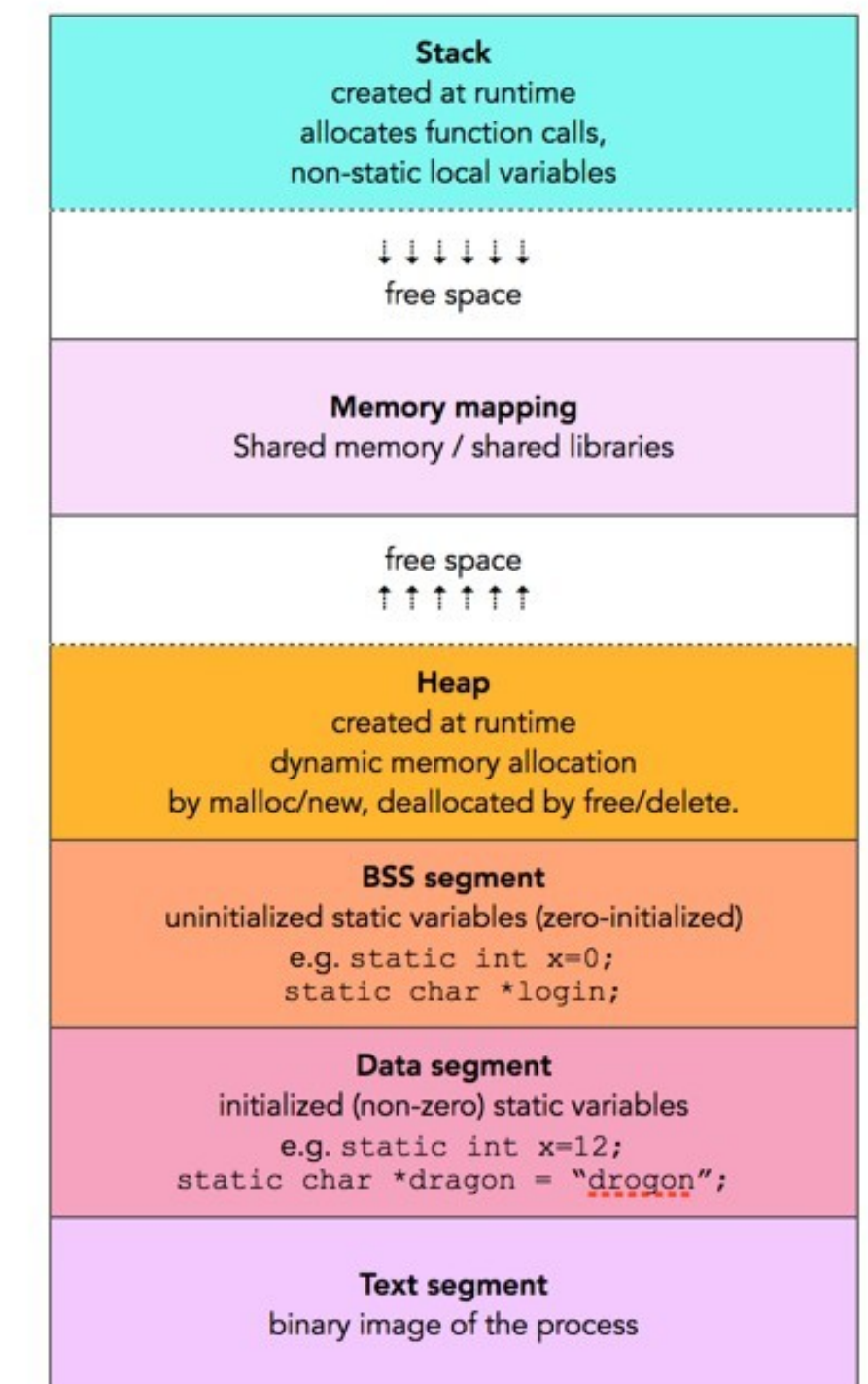
Global or static variables with explicit initial values.

```
int x = 5;    // in .data
static int y = 10;
```

2. **.bss** — Uninitialized Global/Static Variables

- Global or static variables without explicit initialization.
- Physically smaller because uninitialized variables need not be stored in the binary:

```
int counter;  // in .bss (zero-initialized)
static int flag;
```



Heap

This area is reserved for dynamically allocated memory—space that a program requests and frees while it runs (such as with malloc/free in C/C++ or new/delete in C++). The heap typically expands upward toward higher memory addresses.

- Region used for dynamic memory allocation.
- Grows upward (toward higher memory addresses) in many architectures.
- Managed via:
 - > malloc/free in C
 - > new/delete in C++
 - > Box, Vec, etc. in Rust
 - > Garbage collectors in languages like Java, Go, Python
- Characteristics:
 - > Long-lived objects
 - > Manual memory management or garbage collection
 - > Flexible but can lead to fragmentation

Stack

Stack Segment: This segment holds local variables, function arguments, and return addresses associated with function calls. It follows a Last-In, First-Out (LIFO) structure. Whenever a function is invoked, a new stack frame—containing its local data—is pushed onto the stack, and when the function finishes, that frame is removed. The stack usually expands downward toward lower memory addresses, opposite the direction of the heap's growth.

- Grows downward (toward lower memory addresses) on most systems.
- Used for:
 - > Function call frames
 - > Local (automatic) variables
 - > Return addresses
 - > Saved registers

```
void foo() {  
    int a = 10;    // stored on stack  
    int b[100];    // also on stack (unless compiler moves to heap)  
}
```

Characteristics:

- Fast allocation and deallocation (push/pop)
- Limited size
- Automatically cleaned up on function return

Memory leaks and dangling-pointer dereferences and their solutions.

Memory Leaks: A memory leak happens when allocated memory isn't released, gradually draining system resources. A dangling pointer occurs when a pointer still refers to memory that has already been freed. These issues can be prevented by properly freeing memory, setting pointers to NULL, using smart pointers or garbage-collected languages, and managing resources carefully.

Problem: Allocated memory is not freed, gradually reducing available memory.

Solutions:

- Explicitly match every malloc() with free() and new with delete.
- Use garbage-collected languages to reclaim unused memory automatically.
- Use smart pointers or reference counting to manage object lifetimes.
- Apply profiling tools to detect leaks during development.

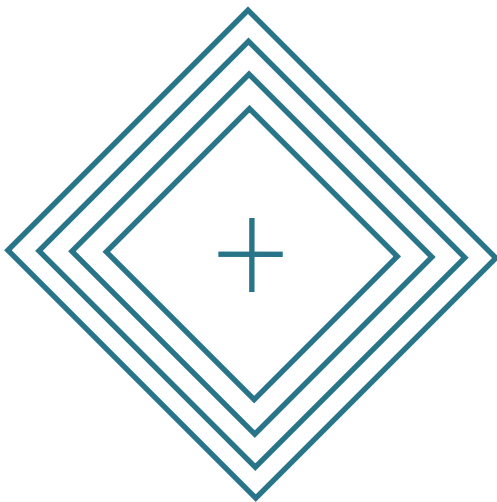
Dangling Pointers: A dangling pointer occurs when a pointer still references memory that has been freed or is no longer valid, leading to undefined behavior or crashes.

Problem: A pointer references memory that has been freed or gone out of scope, causing undefined behavior or crashes.

Solutions:

- Set pointers to NULL after freeing memory.
- Avoid using pointers after the memory they reference is freed.
- Ensure pointers do not reference local variables that go out of scope.
- Initialize pointers to NULL when declared.
- Use languages or features (e.g., Rust) that detect dangling





Memory Leaks vs. Dangling Pointer Dereferences

— Comparison Table —



ASPECT	MEMORY LEAKS	DANGLING POINTER DEREFERENCES
DEFINITION	OCCUR WHEN MEMORY IS ALLOCATED BUT NEVER RELEASED, GRADUALLY CONSUMING AVAILABLE MEMORY.	OCCUR WHEN A POINTER REFERENCES MEMORY THAT HAS ALREADY BEEN FREED OR HAS GONE OUT OF SCOPE.
PRIMARY CAUSE	FORGETTING TO DEALLOCATE MEMORY AFTER USE.	USING A POINTER AFTER ITS TARGET MEMORY HAS BEEN DEALLOCATED OR HAS EXPIRED.
SYMPTOMS	INCREASING MEMORY USAGE OVER TIME; POTENTIAL SYSTEM SLOWDOWN OR CRASHES.	UNDEFINED BEHAVIOR, CRASHES, OR CORRUPTED DATA WHEN DEREFERENCING THE POINTER.
EXAMPLE SCENARIO	<code>MALLOC()</code> OR <code>NEW</code> IS CALLED BUT THE CORRESPONDING <code>FREE()</code> OR <code>DELETE</code> IS NEVER EXECUTED.	A FUNCTION RETURNS A POINTER TO A LOCAL VARIABLE, OR A POINTER IS USED AFTER CALLING <code>FREE()</code> OR <code>DELETE</code> .
RISK LEVEL	TYPICALLY LONG-TERM DEGRADATION OF PERFORMANCE AND STABILITY.	IMMEDIATE AND SEVERE ERRORS, INCLUDING CRASHES AND DATA CORRUPTION.
SOLUTIONS (C/C++)	• MATCH EVERY <code>MALLOC()</code> WITH <code>FREE()</code> AND EVERY <code>NEW</code> WITH <code>DELETE</code> • USE SMART POINTERS (E.G., <code>STD::UNIQUE_PTR</code>, <code>STD::SHARED_PTR</code>) • USE TOOLS AND PROFILERS TO DETECT LEAKS • PREFER LANGUAGES WITH GARBAGE COLLECTION WHEN POSSIBLE	• SET POINTERS TO <code>NULL</code> AFTER FREEING MEMORY • AVOID DEREFERENCING POINTERS AFTER <code>FREE()</code> OR <code>DELETE</code> • DO NOT RETURN POINTERS TO LOCAL VARIABLES • INITIALIZE POINTERS TO <code>NULL</code> WHEN DECLARED
SOLUTIONS (OTHER LANGUAGES)	• GARBAGE-COLLECTED LANGUAGES (JAVA, PYTHON, GO) RECLAIM UNUSED MEMORY AUTOMATICALLY • REFERENCE COUNTING STRATEGIES	• USE LANGUAGES OR FEATURES WITH STRICT OWNERSHIP MODELS (E.G., RUST) THAT PREVENT DANGLING POINTERS AT COMPILE TIME
GOAL	ENSURE MEMORY IS PROPERLY RELEASED WHEN NO LONGER NEEDED.	ENSURE POINTERS ONLY REFERENCE VALID MEMORY.

Garbage Collection

Garbage Collection (GC) automatically manages memory by reclaiming objects no longer in use, relieving developers from manual deallocation. It identifies reachable objects, frees unreachable ones, and helps prevent memory leaks while improving program efficiency.

How Garbage Collection Works

- **Identification:** Finds objects no longer referenced by the program.
- **Reclamation:** Frees memory from unused objects, using methods like copying or mark-and-sweep.
- **Management:** Allocates new memory to the program and returns unused memory to the system.

Key Benefits of Garbage Collection

- **Simplifies programming:** Frees developers from manually managing memory.
- **Prevents errors:** Avoids memory leaks and invalid memory access.
- **Improves performance:** Reclaims unused memory, helping applications run efficiently and smoothly.

Potential Considerations of Garbage Collection

- **Stop-the-world pauses:** GC may temporarily halt the application to clean up memory.
- **Performance tuning:** Understanding GC behavior is important for optimizing performance.
- **GC modes:** Different systems offer modes or controls, such as incremental collection or manual triggers.

